12. To Create Pipes, Redirect Streams, Execute Connected Commands, Close Descriptors, Synchronize Processes, Test Data Flow.

Code:

```
  GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <string.h>

int main() {

    int pipefd[2];
    pid_t pid1, pid2;

    /*
     * pipefd[0] -> read end
     * pipefd[1] -> write end
     */

    printf("Pipe Communication System\n");
    printf("-------------------------\n");

    /* Create pipe */
    if (pipe(pipefd) == -1) {
        perror("pipe");
        return 1;
    }

    printf("Pipe created successfully.\n");

    /* Create first child */
    pid1 = fork();

    if (pid1 < 0) {
        perror("fork");
        return 1;
    }

    if (pid1 == 0) {

        /*
         * First child:
         * Executes "echo Hello World"
         */

        printf("Child 1: Sending data through pipe.\n");

        /* Close unused read end */
        close(pipefd[0]);

        /* Redirect stdout to pipe */
        dup2(pipefd[1], STDOUT_FILENO);

        /* Close duplicated descriptor */
        close(pipefd[1]);
```

```
  GNU nano 8.7.1
        /* Close duplicated descriptor */
        close(pipefd[1]);

        execlp("echo", "echo", "Hello World", NULL);

        /* Runs only if exec fails */
        perror("exec echo");
        exit(1);
    }

    /* Create second child */
    pid2 = fork();

    if (pid2 < 0) {
        perror("fork");
        return 1;
    }

    if (pid2 == 0) {

        /*
         * Second child:
         * Executes "wc -w"
         */

        printf("Child 2: Receiving data from pipe.\n");

        /* Close unused write end */
        close(pipefd[1]);

        /* Redirect stdin from pipe */
        dup2(pipefd[0], STDIN_FILENO);

        /* Close duplicated descriptor */
        close(pipefd[0]);

        execlp("wc", "wc", "-w", NULL);

        /* Runs only if exec fails */
        perror("exec wc");
        exit(1);
    }

    /* Parent process */

    printf("Parent: Closing unused pipe descriptors.\n");

    /* Parent does not use the pipe */
    close(pipefd[0]);
    close(pipefd[1]);

    /* Wait for both children */
```

```
    printf("Parent: Closing unused pipe descriptors.\n");

    /* Parent does not use the pipe */
    close(pipefd[0]);
    close(pipefd[1]);

    /* Wait for both children */
    waitpid(pid1, NULL, 0);
    waitpid(pid2, NULL, 0);

    printf("Parent: Both child processes completed.\n");
    printf("Data flow completed successfully.\n");

    return 0;
}
```

Result:

```
bheeshma@SINGAM:~/SKILL$ nano Sk          ill12a.c
bheeshma@SINGAM:~/SKILL$ gcc Ski          ll12a.c -o s12a
bheeshma@SINGAM:~/SKILL$ ./s12a
Pipe Communication System
-------------------------
Pipe created successfully.
Child 1: Sending data through pipe.
Child 2: Receiving data from pipe.
Parent: Closing unused pipe descriptors.
2
Parent: Both child processes completed.
Data flow completed successfully.
bheeshma@SINGAM:~/SKILL$
```

To Support Multiple Pipes, Manage Descriptor Arrays, Launch Multiple Processes, Coordinate Execution, Cleanup Resources, Test Long Pipelines.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <string.h>

#define MAX_COMMANDS 5
#define MAX_ARGS 20

int main() {

    /*
     * Pipeline:
     *
     * echo Hello World
     *          |
     *          v
     * tr a-z A-Z
     *          |
     *          v
     * tr ' ' '\n'
     *          |
     *          v
     * wc -l
     */

    char *commands[MAX_COMMANDS][MAX_ARGS] = {
        {"echo", "Hello World", NULL},
        {"tr", "a-z", "A-Z", NULL},
        {"tr", " ", "\n", NULL},
        {"wc", "-l", NULL}
    };

    int commandCount = 4;

    /* Pipe descriptor array */
    int pipes[MAX_COMMANDS - 1][2];

    pid_t pids[MAX_COMMANDS];

    printf("Multiple Pipeline Execution\n");
    printf("---------------------------\n");

    printf("Pipeline:\n");

    for (int i = 0; i < commandCount; i++) {

        printf("%s", commands[i][0]);

        for (int j = 1; commands[i][j] != NULL; j++)
            printf(" %s", commands[i][j]);
        }
```

```
 GNU nano 8.7.1
        for (int j = 1; commands[i][j] != NULL; j++) {
            printf(" %s", commands[i][j]);
        }

        if (i < commandCount - 1)
            printf(" | ");
    }

    printf("\n\n");

    /* Create all required pipes */
    for (int i = 0; i < commandCount - 1; i++) {

        if (pipe(pipes[i]) == -1) {

            perror("pipe");

            return 1;
        }
    }

    printf("Created %d pipes.\n", commandCount - 1);

    /* Create processes */
    for (int i = 0; i < commandCount; i++) {

        pids[i] = fork();

        if (pids[i] < 0) {

            perror("fork");

            return 1;
        }

        /* Child process */
        if (pids[i] == 0) {

            /*
             * If not the first command,
             * read from previous pipe.
             */
            if (i > 0) {

                dup2(pipes[i - 1][0], STDIN_FILENO);
            }

            /*
             * If not the last command,
             * write to next pipe.
             */
            if (i < commandCount - 1) {
```

```
 GNU nano 8.7.1
            /*
             * If not the last command,
             * write to next pipe.
             */
            if (i < commandCount - 1) {

                dup2(pipes[i][1], STDOUT_FILENO);
            }

            /*
             * Close all pipe descriptors
             * after redirection.
             */
            for (int j = 0;
                 j < commandCount - 1;
                 j++) {

                close(pipes[j][0]);
                close(pipes[j][1]);
            }

            /* Execute command */
            execvp(commands[i][0], commands[i]);

            /* Execute only if exec fails */
            perror("execvp");
            exit(1);
        }
    }

    /*
     * Parent closes all pipe descriptors.
     */
    for (int i = 0;
         i < commandCount - 1;
         i++) {

        close(pipes[i][0]);
        close(pipes[i][1]);
    }

    printf("Created %d child processes.\n",
           commandCount);

    printf("Waiting for processes to complete...\n\n");

    /* Wait for all children */
    for (int i = 0; i < commandCount; i++) {

        waitpid(pids[i], NULL, 0);
    }
```

```c
     * Parent closes all pipe descriptors.
     */
    for (int i = 0;
         i < commandCount - 1;
         i++) {

        close(pipes[i][0]);
        close(pipes[i][1]);
    }

    printf("Created %d child processes.\n",
           commandCount);

    printf("Waiting for processes to complete...\n\n");

    /* Wait for all children */
    for (int i = 0; i < commandCount; i++) {

        waitpid(pids[i], NULL, 0);
    }

    printf("\nAll processes completed.\n");
    printf("All pipe resources cleaned up.\n");
    printf("Pipeline execution completed successfully.\n");

    return 0;
}
```

Result:

```
bheeshma@SINGAM:~/SKILL$ nano Sk          ill12b.c
bheeshma@SINGAM:~/SKILL$ gcc Ski          ll12b.c -o s12b
bheeshma@SINGAM:~/SKILL$ ./s12b
Multiple Pipeline Execution
---------------------------
Pipeline:
echo Hello World | tr a-z A-Z | tr
 | wc -l

Created 3 pipes.
Created 4 child processes.
Waiting for processes to complete...

2

All processes completed.
All pipe resources cleaned up.
Pipeline execution completed successfully.
bheeshma@SINGAM:~/SKILL$
```